

My company AI Journey

**From Manual Coding to ChatGPT,
Cursor, and Claude Code**

About me

KOWA Jia Liang

Senior Software Engineer (Singapore) - Payboy PTE LTD

- ❖ Payboy is a product based company, and they do HRMS SaaS product, helping companies handling like payrolls, leaves, attendance, taxes, etc
- ❖ We mainly using Ruby on Rails for backend, Vue JS for frontend.
- ❖ My day to day tasks will be mostly on planning tasks with PM / reviewing PR

Evolution of AI in Our Workspace

1. Manual coding

Traditional syntax typing.
Browsing through
documentation and
StackOverflow pages
manually for solutions.

3. Cursor IDE

Finally have AI Agent sit
within IDE, fast responses,
indexing files & give
autocompletes
recommendation.

2. Prompting (ChatGPT)

Copy-pasting code fragments to
web chats for opinions & answers.
Fast speed but it's totally context
blindness.

4. VS Code + Claude (currently using)

Very similar to Claude, but
abit different, it has
autonomous background
agent loops running checks,
tests, and diagnostics locally.

Two different AI era: Browser vs. Code Editor

❖ Browser AI (The Outside Consultant)

- ❖ Repeat copying, pasting, and switching windows to get solutions.
- ❖ AI has Zero Context: Requires manually describing imports, types, helpers, and more.

❖ Editor-Native AI (The Resident Partner)

- ❖ With this, NO more copy & paste & switching browser, AI lives directly in your workspace to read/write files.
- ❖ Deep Awareness: Background indexing of files, utilities, and styling standards, so you don't need to repeat what you using, what's the product logic every time.

**How we use Claude Code to
improve our team performance**

Automate processes with Claude Code

Permanent Context (CLAUDE.md)

Is a file that storing standardises active tech stacks for project, strict build sequences, testing parameters, and styling rules, placed directly at the project root.

Custom Skills

Create custom repository workflows inside `.claude/skills/`, triggered instantly via sidebar slash commands.

E.g. `/changelog`

MCP (USB-C for AI apps)

A standardized interface for AI agent, we can use MCP to connects to various of **data sources and tools**, enabling the AI to retrieve or post data easily.

***Still WIP**, but I already have Proof of concept.

Our Claude permanent context

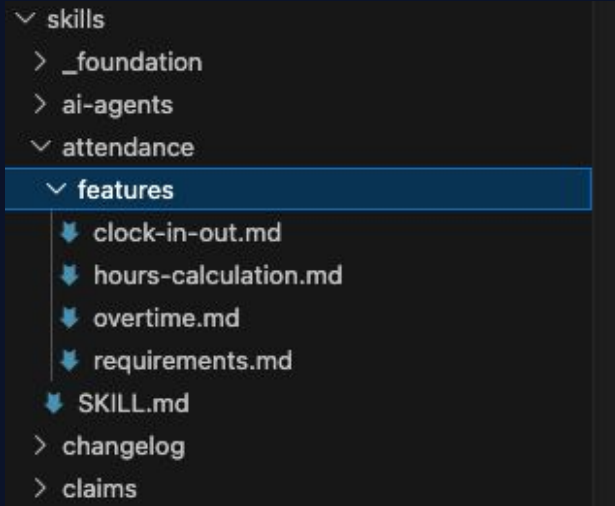
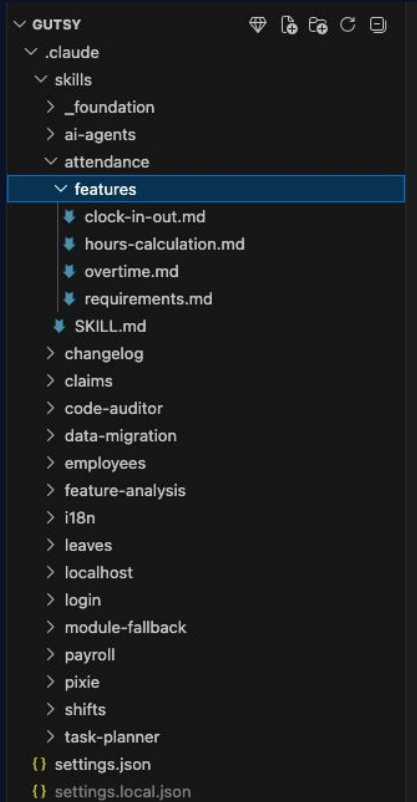
```
CLAUDE.md x
# CLAUDE.md > *# # Gutsy HR/Payroll Platform > *# # Service Module Pattern (Frontend)
You, yesterday | 4 authors (Zainudin Sainal and others)
1 # Gutsy HR/Payroll Platform
2
3 > ## Project Architecture--
10
11 ## Frontend Architecture
12
13 > ### Three Portals--
30
31 > ### Data Flow--
36
37 > ### Vite-Ruby Integration--
67
68 > ## Critical Code Review Checklist--
108
109 ## Performance Patterns Quick Reference
110
111 > ### Controllers - Always Eager Load--
125
126 > ### Services - Pre-fetch Hash Tables -
145
146 > ### Batched WHERE Clauses - Avoid Per-Item Queries--
161
162 > ### API Endpoints - Use Column Aliasing--
173
174 > ## Enum Collections Pattern--
211
212 ## Rails API Controller Patterns
213
214 > ### Initial Load Pattern--
228
229 > ### API Response Patterns--
248
249 > ### Best Practices--
255
```

Permanent Context

This is the initial file placed in the project root, which will auto loads the apps logic context (we call it knowledge also for easier understanding among our team)

As you can there is a lot of ###, those are like a header, then we specify our requirement like apps overview logics, code structure, and more within them

Custom skills



Custom Skills

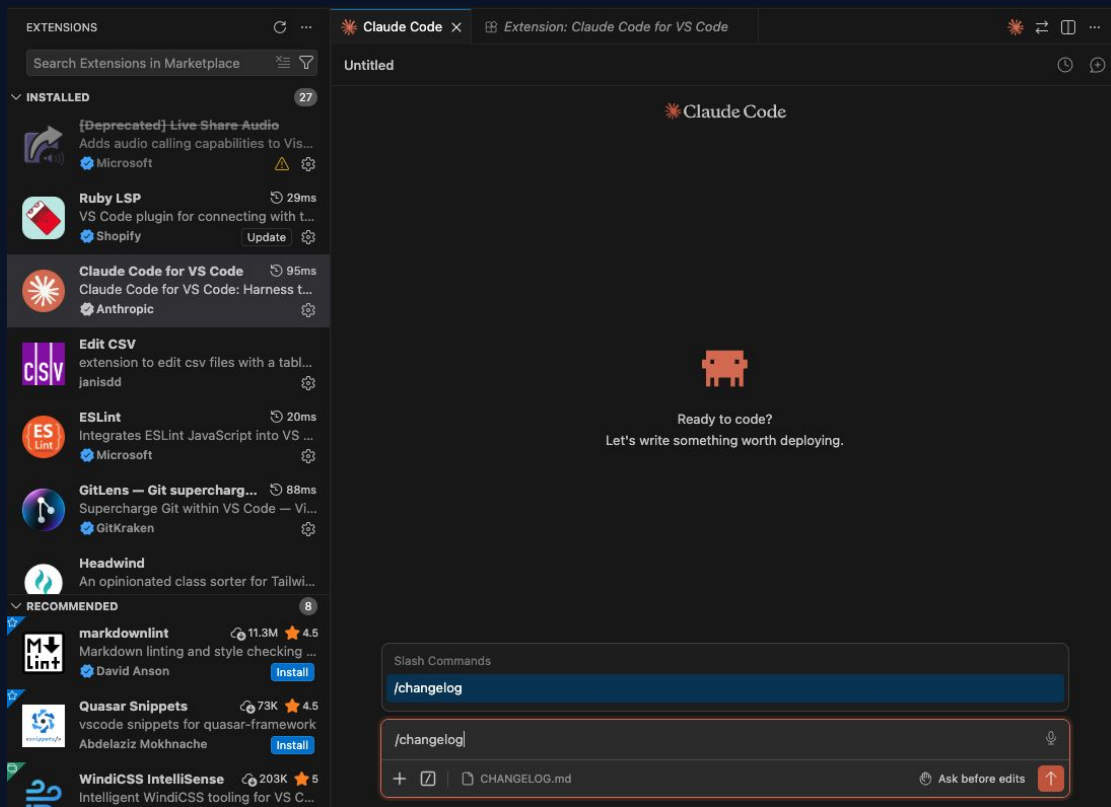
These are the skills in our repo, each repo is representing as an individual skills.

Each skills repo has a `SKILL.md` file, that tells the Agent when to trigger it (can be manual / auto).

As of the `features` repo (this is optional), what it does is to stores our product logic for the skill.

Case study for custom skills

Case study 1: Custom skill `/changelog`



Tracking changelog

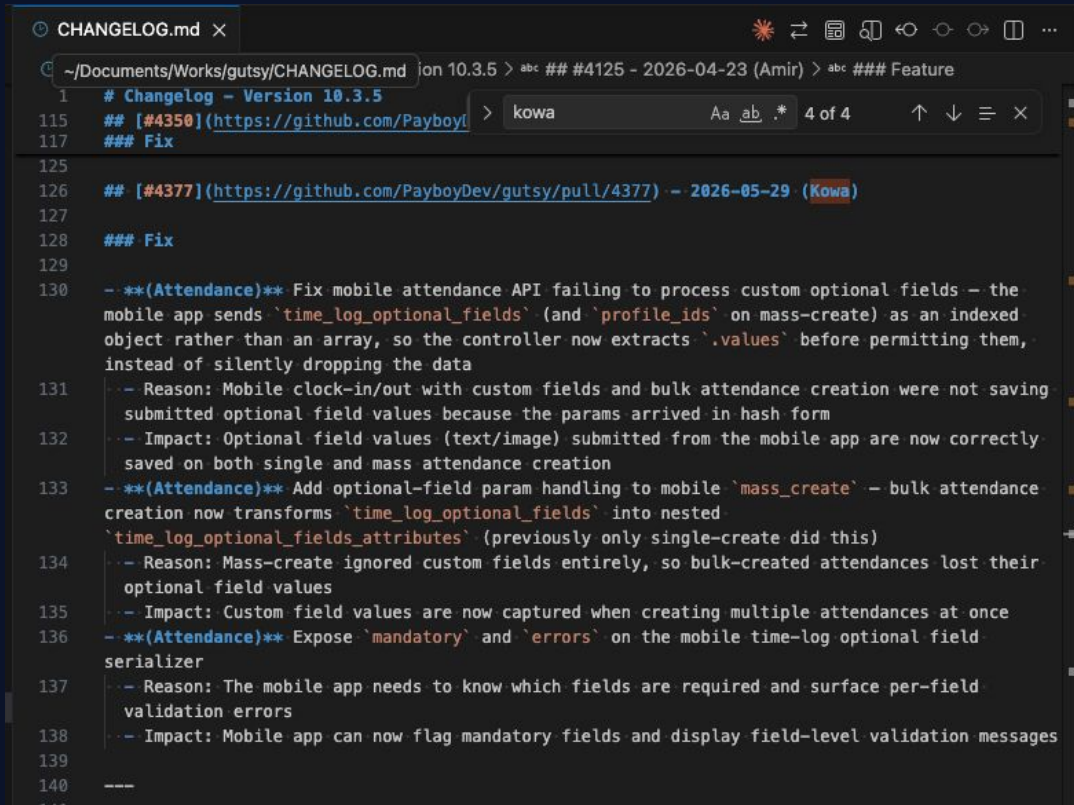
For us we do changelog to record down what we did for each PR that get merges into staging

And sometime it's hard to summary what we have did for our PR.

Like for example, I fixed a bug, but what is a bug and how to describe, sometime is just time consuming to log this down.

So to solve this issue, I created this `/changelog` skill.

Case study 1: Custom skill /changelog cont.



```
CHANGELOG.md x
~/Documents/Works/gutsy/CHANGELOG.md  jon 10.3.5 > abc ## #4125 - 2026-04-23 (Amir) > abc ### Feature
1 # Changelog - Version 10.3.5
115 ## [#4350](https://github.com/PayboyL) > kowa Aa ab .* 4 of 4 ↑ ↓ ≡ ×
117 ### Fix
125
126 ## [#4377](https://github.com/PayboyDev/gutsy/pull/4377) -- 2026-05-29 (Kowa)
127
128 ### Fix
129
130 --**(Attendance)** Fix mobile attendance API failing to process custom optional fields - the
mobile app sends `time_log_optional_fields` (and `profile_ids` on mass-create) as an indexed
object rather than an array, so the controller now extracts `.values` before permitting them,
instead of silently dropping the data
131 -- Reason: Mobile clock-in/out with custom fields and bulk attendance creation were not saving
submitted optional field values because the params arrived in hash form
132 -- Impact: Optional field values (text/image) submitted from the mobile app are now correctly
saved on both single and mass attendance creation
133 --**(Attendance)** Add optional-field param handling to mobile `mass_create` - bulk attendance
creation now transforms `time_log_optional_fields` into nested
`time_log_optional_fields_attributes` (previously only single-create did this)
134 -- Reason: Mass-create ignored custom fields entirely, so bulk-created attendances lost their
optional field values
135 -- Impact: Custom field values are now captured when creating multiple attendances at once
136 --**(Attendance)** Expose `mandatory` and `errors` on the mobile time-log optional field
serializer
137 -- Reason: The mobile app needs to know which fields are required and surface per-field
validation errors
138 -- Impact: Mobile app can now flag mandatory fields and display field-level validation messages
139
140 ---
141
```

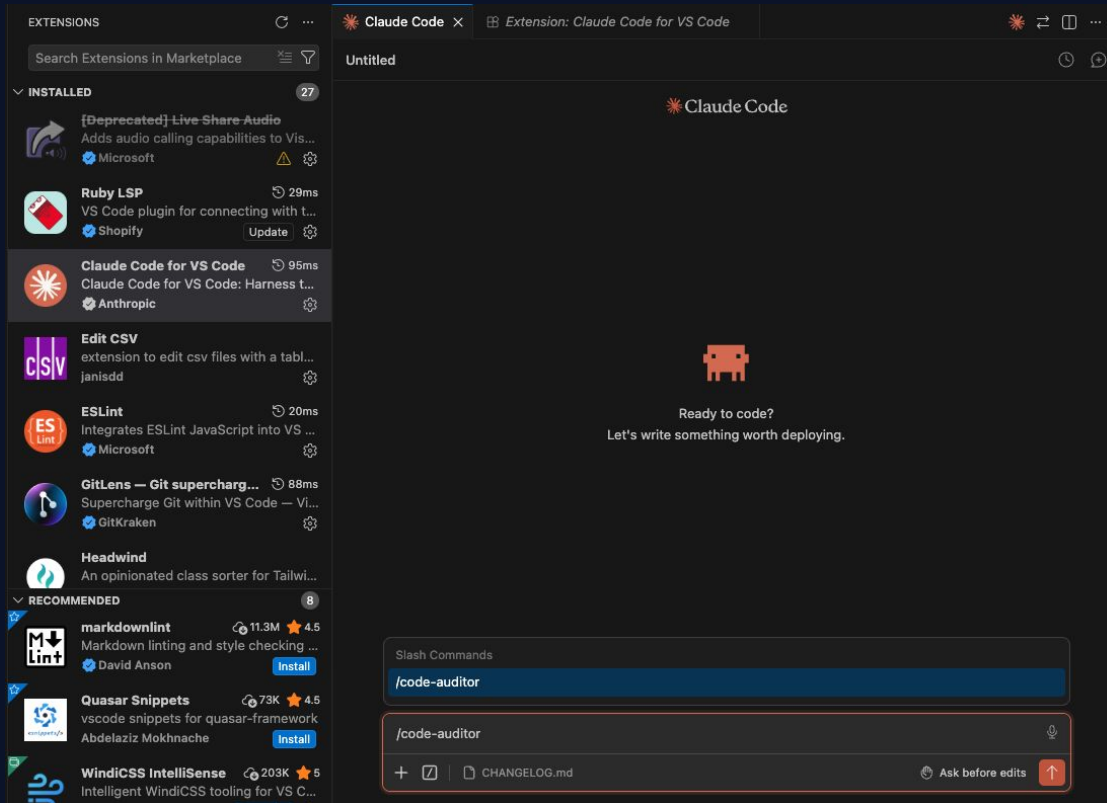
Result running `/changelog`

After trigger, Agent will read my changes (current branch VS target branch), then summary what i had done, so I can skip thinking what I need to log into changelog.

It list down what module I touch, the reason of this changes, and what is the impact etc.

I can add in more things like Jira card link / GitScrum card link or whatever you like.

Case study 2: Code-auditor skills (My favorite)



Code-auditor skills (PR Review tools)

For me because I am in-charge to do PR review for my teams, so I've been always repeating the same comments to the developers, so instead of repeating all the comments, I came up this skills **`code-auditor`**.

So what it does is to help me do a pre pr-review, I ask my devs to run this skill before they send to me for pr review, and with this, it wipes out a lot comments for me, so I only need to review on the logic side (which saves me a lot of times)

Case study 2: Code-auditor skills cont.

Step 1: Gather context

`code-auditor` will request Agent to ask the user: Which branch you wants to compare with?

Default to staging, but we also can input other branch to proceed.

Step 2: Run PR review

This step will run the actual PR review as per my **`code-auditor`** rules.

Also whatever modules you updated, it will also auto-trigger the relates skills (knowledge), to make sure AI agent understand the product logic correctly, so they can give valid & correct inputs.

Step 3: Review done

After review finish, it will output a `.md` file with all the concerns (label from critical, important, and suggestion).

Also the listed concern will come with suggested solutions too, so before I do actual PR review, the skills already done 70-80% for me already.

Show in VS-code for a after run example.

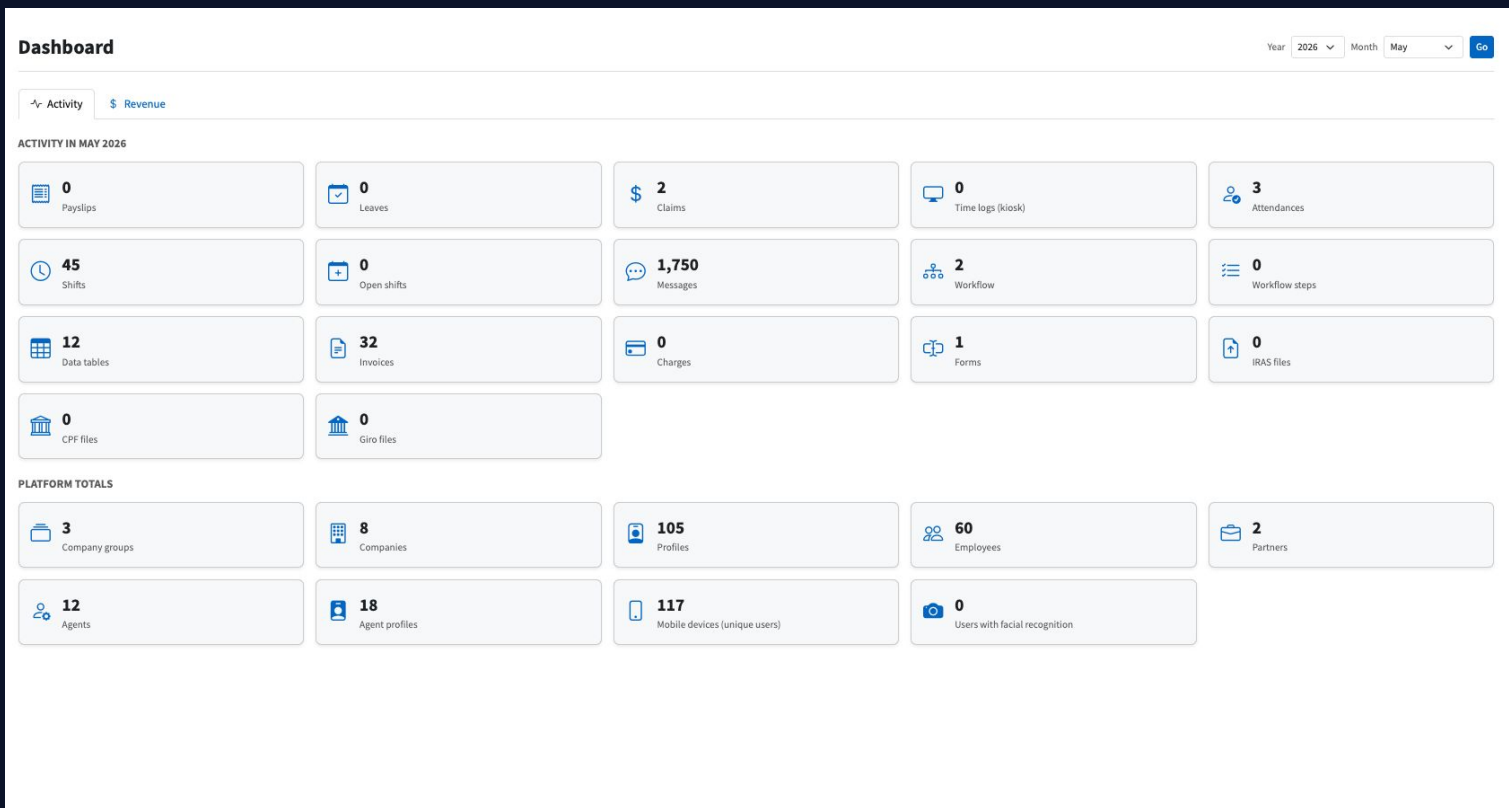
Case study 3: Superpower plugin

- ❖ Whenever our team trying to do a new feature, normally we will use this **`superpower plugin`**.
- ❖ **`Superpower`** is a software development methodology for coding agent, its a public repo on github
- ❖ How it work:
 - When you trigger this superpower, first he will digest your requirement (from chat), then they will give you a proposal, or we can say its a coding plan with details like what files he will be creating (according to your coding standard rules), come up with core logic functions with test cases in a .md file.
 - After you reviewed the .md file, you can chat and make any changes you want, then you sign off (to give green light to agent & proceed).
 - Finally, Claude Agent will follow the .md design file to start coding & designing, then Claude will create parallel agents to test the written code, fix bug & test & repeat until all good.

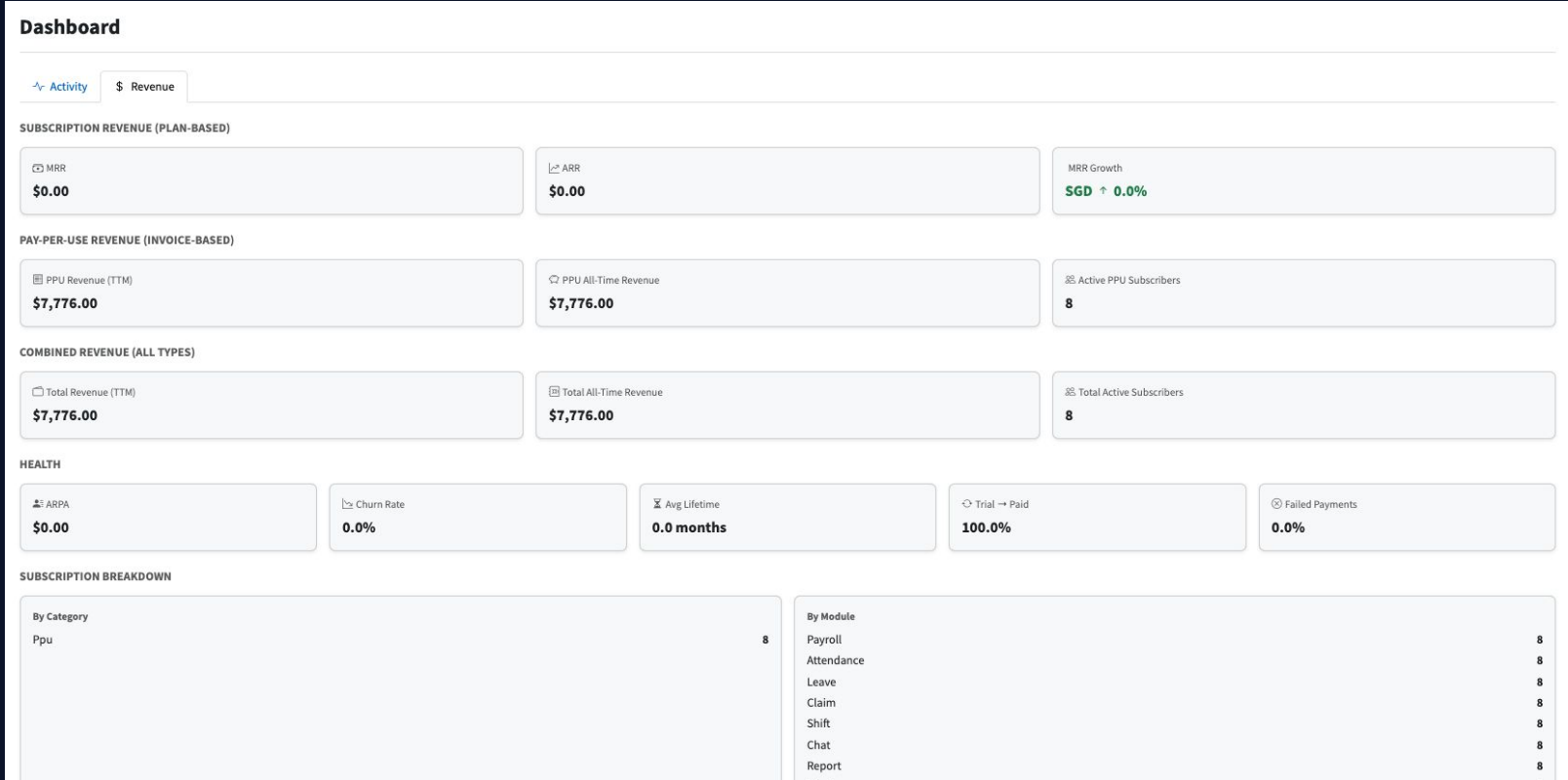
Case study 3: Superpower plugin cont.

- ❖ An example of `superpower` work, in my company system, we have a backend control system for our datas, we always wanted to do a report pages to know what is our Monthly Recurring Revenue (MRR), also usage for each modules per month / per year. But due to no time to invest to this, so the project was delay for months.
- ❖ So I used `superpower` and prompt
 - Refer to <https://github.com/rameereez/profitable> , help me create something similar to this in my backend control system page, also i want a usage report for each of my modules.
- ❖ Then it start planning for around 8 mins - 10 mins, back to back checking with me the requirements for this feature.
- ❖ After that it tooks around 15 mins to finish the feature.
- ❖ My PR: <https://github.com/PayboyDev/gutsy/pull/3922/changes>
- ❖ To me, it is good to come up with something first, at least prototype, then i can fix from their implementation to avoid a lot easy job like frontend UI.

Case study 3: Superpower plugin cont.



Case study 3: Superpower plugin cont.



Model Context Protocol (MCP)

MCP is a standardized interface for AI agent, we can use MCP to connects to various of **data sources and tools**, enabling the AI to retrieve or post data easily.

❖ The Old Way (Custom Cable Mess)

- ❖ API Chaos: Writing custom API code for every external connection.
- ❖ Fragile: Integrations easily break whenever the tool API structures change.

❖ The MCP Way (The Universal USB)

- ❖ One Port: Exposes tool capability natively directly inside Claude.
- ❖ Plug-and-Play: Zero custom integration glue code required. Easy setup thru jsonb file

My next move: Connecting Claude to Sentry (error logging applications)

Connecting Sentry to our Claude allows it to instantly access production stack traces and error contexts without me having to manually copy-paste logs. This enables the agent to provide automated, accurate diagnoses and suggest precise fixes directly within the development environment

Phase	Before (The 'Hunting' Phase)	After (The 'Guided' Phase)
Search	Hunting through raw metrics and diagnostic dashboards (30m+).	Clicking the alert link; Claude immediately maps and opens local file.
Guess	Attempting to author a fix matching current patterns manually.	Claude suggests an isolated, standards-compliant patch matching CLAUDE.md.
Verify	Locally running tests inside terminal suites manually.	Claude invokes local test commands to validate stability before commit.

Why We Moved: Cursor vs. Claude

Understand that Cursor & Claude actually behave quite similar, but in the end we still choosing claude for our teams is because of we are product based company, so our product logic are customized and heavy, so in the end we choose quality over speed.

Metric	Cursor (The Typist IDE)	Claude (The System Architect)
Active Chat Context	Chat Pollution: Fills chat window with code blocks, leading to slow replies. Whenever you doing a big feature, the response will not be too accurate	Context Isolation: Spawns parallel background subagents to work behind the scenes & give answer only to the origin chat agent.
Execution Logic	Inline focus: Predicts autocomplete steps dynamically while typing. (Very good for inline logic & redundant codes)	Slower, no auto-complete suggestion
Task Size & Speed	Fast Response: Near-instant response. Perfect for small prompt sizes, quick debugging, and micro-queries.	Slower but Smarter: Takes longer to process, but the resulting logic is significantly more accurate.

Questions & Answers

Thank you for your time.